

Identification of Segments (polygon edges) in Moser's Circle.

Moser's circle has some number of points on its perimeter. A chord connects every point to every other point. However, the points are arranged such that no three chords ever concur at any internal point. Which is to say that three or more chords never intersect in the interior of the circle. This has been proven to always be true for an odd number of points that are symmetrically distributed about the circle. It is trivially false for an even number of points likewise distributed. Although Moser actually worded his problem to avoid symmetric distribution if it would result in concurrent internal intersections. He suggested moving the points about to maximize the number of internal polygons created as a result of the intersections.

How many chords are there? We might build a sequence and look for a pattern.

Number of Perimeter Points, n	Number of chords
2	1
3	3
4	6
5	10
6	15
7	21

So, I see the pattern quickly enough. The number of chords is the triangular number which corresponds to the number of points: number of chords = $n(n-1)/2$. Now, from n, can we figure out how many intersections might there be? Well actually, yes. For every group of four distinct perimeter points, there must be exactly one intersection because every point is connected to all other points. Thus any group of four makes a quadrilateral and the interior chords must cross. Since order doesn't matter, the permutations of n points taken four at a time is given by $\{n(n-1)(n-2)(n-3)/4!\}$.

Looking at the drawing for n=5, we can see that the intersections make some number of planar regions, where a region is a white space containing no lines and no points. Indeed for every region it is outlined by edges and its corners are the intersections. If we put a new point at each intersection, then those become vertices of the regions. The possible exception is that some regions are defined by arcs of the original circle as well as edges and vertices. There is a way to count the number of regions as well. It is given by the formula $n(n-1)/2 + n(n-1)(n-2)(n-3)/4! + 1$. I won't bother to explain the combinatorics to arrive at that. It can be found online. For n=5, there are 16 such regions. Count carefully.

Knowing the number of regions is nice, but how might we collect the vertices of each region as a list. In this case, all of the regions would be a list of vertex lists. A single region would be given as $R=(x_1,y_1),(x_2,y_2),(x_3,y_3), \dots$. If we went around the region ccw, we would shortly return to the starting pair and could say that R is defined by that collection of pairs. If we could get all 16 of the regions written similarly, we could sort them lexicographically and proclaim that they were identified. In some cases the number of pairs around a region would be less than other regions. We would need to make a rule that shorter pair lists sort before longer pair lists or perhaps some other rule, but since none of the regions would have the same pairs, they could be sorted. The arc containing regions are exceptions, but since each will have only two vertices they should be easy to accommodate,

We have yet to answer how to collect the vertices of each region. We stated how to do one region, with some hand waving to suggest that we knew how to follow a road around the region ccw and thus find all of its vertices. But what is the road? The regions if and only if we adhere to Moser's rules will have n or less bounding edges. If we don't adhere to Moser, and allow symmetric perimeter points, then some multiple segments can concur at a single point. Fact: All intersections have occurred along the original chords and we identify those by their two endpoints. Therefore, we could create "roads" = "region edges" by traveling along each chord and every time we come to an intersection, create an edge back to the most recent point from which we came. From this large list of "roads" we could match them with the list of vertices such that we would know all of the roads leading away from every vertex. If we include the perimeter points into the vertex set, then all regions will be bounded by vertices or vertices plus an arc. Now, we could in fact go ccw around each and every vertex to define a region. There would be a lot of redundancy, but we could do it. With respect to redundancy, if a region is surrounded by say five vertices, then that region would get identified five times.

So how to reduce redundant region identification? Each edge can be adjacent to at most two regions and almost always exactly two regions. In Moser's problem, the only exceptions are the arcs, which are adjacent to only one region. Therefore, before we start down a road, we should check a regions list to see if that road is in the list. If it is in the regions list and if we are going along it in the same direction that it was put into the list, then we have already identified that region and we can "go somewhere else". That "somewhere" is either the next road from the current vertex, or if all "roads" from that vertex have been traversed, then we go to the next vertex. Continue until 1) all vertices have been examined, including all edges emanating from each, or 2) the number of identified regions matches the calculated number of regions.